Sorting

March 14, 2024

Selection Sort

- Say, the elements to sort occupy positions 0 to $n-1$ of an array.
- for $i = 0$ to $n-2$
  - /* The elements in positions upto $i-1$ are sorted. */
  - Find the smallest among the elements in positions i to $n-1$.
  - Swap it with the i-th element.

Time taken $= n + (n-1) + \ldots + 2 = O(n^2)$.

Selection Sort - Example Walkthrough

Initial Array: [64, 25, 12, 22, 11]
Iteration 1: Select 11 (min) and swap with the 1st element.
Array: [11, 25, 12, 22, 64]
Iteration 2: Select 12 (min) and swap with the 2nd element.
Array: [11, 12, 25, 22, 64]
Iteration 3: Select 22 (min) and swap with the 3rd element.
Array: [11, 12, 22, 25, 64]
Iteration 4: Select 25 (min) and swap with the 4th element.
Sorted Array: [11, 12, 22, 25, 64]

Selection Sort: C code

```
#include <stdio.h>

void swap(int v[], int i, int j) {
    int temp;
    temp = v[i]; v[i] = v[j]; v[j] = temp;
}

void selectionSort(int arr[], int n) {
    int i, j, minIndex;

    for (i = 0; i < n-1; i++) {
        minIndex = i;
        for (j = i+1; j < n; j++)
            if (arr[j] < arr[minIndex])
                minIndex = j;
        swap(arr, i, minIndex);
    }
}
```

Selection Sort: C code

```
#include <stdio.h>
void printArray(int arr[], int size) {
    int i;
    for (i=0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr)/sizeof(arr[0]);
    printf("Original array: \n");
    printArray(arr, n);
    selectionSort(arr, n);
    printf("Sorted array: \n");
    printArray(arr, n);

    return 0;
}
```

Insertion Sort

- Say, the elements to sort occupy positions 0 to $n-1$ of an array.
- for $i = 1$ to $n-1$
  - /* The elements in positions 0 to $i-1$ are sorted. */
  - Pick the i-th element k out of the array.
  - Scanning right to left, move the previous elements to the right by one position, until an element less than k is reached.
  - Insert k to the right of it.

Time taken $= 1 + 2 + \ldots + (n-1) = O(n^2)$.

Insertion Sort: Example

- Original array : 12, 11, 13, 5, 6
- After the first pass : 11, 12, 13, 5, 6
- After the second pass : 11, 12, 13, 5, 6
- After the third pass : 5, 11, 12, 13, 6
- After the fourth pass : 5, 6, 11, 12, 13

Insertion Sort: C code

```
void insertionSort(int arr[], int n) {
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
```

Insertion Sort: Example

```
int main() {
    int arr[] = {12, 11, 13, 5, 6};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: \n");
    printArray(arr, n);

    insertionSort(arr, n);

    printf("Sorted array: \n");
    printArray(arr, n);

    return 0;
}
```

Bubble Sort

- say, the elements to sort occupy positions 0 to $n-1$ in an array.
- imagine the array vertically with position 0 at the bottom.
- for $i = 0$ to $n-2$
  - /* move the (n-i)-th smallest to the (n-i-1)-st place */
  - for $j = 0$ to $n-i-2$
    - if the j-th element and and the next are out of order, swap them.

Time taken $= (n-1) + (n-2) + \ldots + 1 = O(n^2)$.

Bubble Sort-Example: **Original Array:** 64, 25, 12, 22, 11

		1	2	3	4	5	6	7	8	9	10
4	11	11	11	11	64	64	64	64	64	64	64
3	22	22	22	64	11	11	11	25	25	25	25
2	12	12	64	22	22	22	25	11	11	22	22
1	25	64	12	12	12	25	22	22	22	11	12
0	64	25	25	25	25	12	12	12	12	12	11

Pass 1: Columns 1, 2, 3, 4
Pass 2: Columns 5, 6, 7
Pass 3: Columns 8, 9
Pass 4: Column 10

Bubble Sort - C Code

```
#include <stdio.h>

void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr, j, j + 1);
            }
        }
    }
}
```

Bubble Sort - C Code

```
int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    printArray(arr, n);

    bubbleSort(arr, n);

    printf("Sorted array: ");
    printArray(arr, n);

    return 0;
}
```

Counting Sort

- Sorting algorithm introduced by Harold H. Seward in 1954.
- Operates in linear time.
- Efficient for sorting integers with a known range.

Counting Sort

1. Find the range of input elements.
2. Create an auxiliary array (count array) to store element counts.
3. Traverse the input array and update the count array.
4. Compute the prefix sums over the count array.
5. for $i = n - 1$ downto 0
   - Let j be the i-th input element
   - There are count[j] input elements that are $\leq$ this j; so, this j goes to output location count[j] $- 1$.
   - decrement count[j].

Counting Sort

- Time complexity: $O(n + k)$
- n - number of elements in the array
- k - range of input values
- Stable sort - maintains relative order of equal elements between input and output

Counting Sort

- Advantages:
  - Linear time complexity for suitable cases.
  - Stable sorting.
- Limitations:
  - Practical for small ranges, less practical for large ranges.
  - Limited to integer data.

Example

- Consider the input array: $[4_a, 2_a, 3_a, 1_a, 4_b, 2_b, 3_b, 4_c, 1_b, 2_c]$.
- The subscripts are to distinguish elements with the same key values
- Range: 1 to 4. (subscripts are not part of keys)
- Count array initialization: $[0, 0, 0, 0]$.

Step 1: Count Occurrences

- Traverse the input and update the count array.
- Input array: $[4_a, 2_a, 3_a, 1_a, 4_b, 2_b, 3_b, 4_c, 1_b, 2_c]$.
- Count array: $[2, 3, 2, 3]$.

Step 2: Cumulative Counts

- Compute prefix sums over the count array.
- Prefix sums of the count array: $[2, 5, 7, 10]$.

Step 3: Build Sorted Output

- Use the prefix sums array to build the sorted output array.
- 2_c is the last element; there are 5 that are $\leq 2_c$; so, 2_c goes to output location 4.
- Prefix sums array is $[2, 4, 7, 10]$ now.
- 1_b is the 2nd last element; there are 2 that are $\leq 1_b$; so, 1_b goes to output location 1.
- Prefix sums array is $[1, 4, 7, 10]$ now.
- 4_c is the 3rd last element; there are 10 that are $\leq 4_c$; so, 4_c goes to output location 9.
- Prefix sums array is $[1, 4, 7, 9]$ now.
- ...
- Sorted output array: $[1_a, 1_b, 2_a, 2_b, 2_c, 3_a, 3_b, 4_a, 4_b, 4_c]$.